

BINV2150-A – Javascript : avancé

Séance 14 – Router

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École Léonard de Vinci

MPA vs SPA



- MPA : Multi Page Application
  - Chaque page est un document HTML distinct
  - Le serveur envoie une nouvelle page pour chaque requête
  - Rechargement complet de la page
- SPA : Single Page Application
  - Une seule page HTML
  - Le contenu est mis à jour dynamiquement via JavaScript
  - Navigation sans rechargement complet de la page

Routage à la main



```
const App = () => {
    const [page, setPage] = useState<string>("home");

    return (
        <div>
            <nav>
                <button onClick={() => setPage("home")}>Home</button>
                <button onClick={() => setPage("about")}>About</button>
            </nav>
            <main>
                {page === "home" &&<Home />}
                {page === "about" &&<About />}
            </main>
        </div>
    );
};
```

Problèmes



- Pas d'URL différente pour chaque page
- Pas de navigation avec les boutons "Précédent" et "Suivant" du navigateur
- Gestion manuelle de l'état de la page

Routeur



- Gère la navigation entre différentes pages ou vues d'une application
- Intercepte les changements d'URL et rend le composant correspondant
- Single Page Application (SPA)
- Librairie React Router

Définition des routes



- Configuration dans le fichier `main.tsx`
- Chaque route est définie par :
  - Un chemin (path) : `"/about"`
  - Un composant (element) : `<AboutPage />`
- Le routeur affiche le composant correspondant au chemin actuel

Définition des routes



```
1  import { createBrowserRouter, RouterProvider } from
       "react-router-dom";
2
3  const router = createBrowserRouter([
4      {
5          path: "/",
6          element: <HomePage />,
7      },
8      {
9          path: "/about",
10         element: <AboutPage />,
11     },
12 ]);
13
14 ReactDOM.createRoot(document.getElementById("root")!).render(
15     <React.StrictMode>
16         <RouterProvider router={router}/>
17     </React.StrictMode>
18 );
```

Nested routes



- Routes imbriquées
- Permet de définir une hiérarchie de routes
- Un composant parent peut avoir plusieurs composants enfants
- Utile pour les layouts communs (ex : barre de navigation)

Nested routes

Définition des routes imbriquées



```
const router = createBrowserRouter([
    {
        path: "/",
        element: <App />,
        children: [
            {
                path: "",
                element: <HomePage />,
            },
            {
                path: "about",
                element: <AboutPage />,
            },
            {
                path: "contact",
                element: <ContactPage />,
            }
        ],
    },
]);
```

Nested routes

Outlet



```jsx
import { Outlet } from "react-router-dom";

const App = () => {
    return (
        <div>
            <NavBar />
            <Outlet />
            {/* Sera remplacé par le composant enfant correspondant */}
        </div>
    );
};
```

Navigation entre les routes



- Composant `Link`
  - Utilisable comme la balise HTML `<a>`
  - Moins flexible
- Hook `useNavigate`
  - Fonction utilisable dans un gestionnaire d'évènement
  - Utilisable pour des actions plus complexes
- Permettent de naviguer entre les routes sans recharger la page

Navigation entre les routes

Link



```jsx
import { Link } from "react-router-dom";

const NavBar = () => {
    return (
        <nav>
            <Link to="/">Home</Link>
            <Link to="/about">About</Link>
            <Link to="/contact">Contact</Link>
        </nav>
    );
};
```

Navigation entre les routes

hook useNavigate



```
1  import { useNavigate } from "react-router-dom";
2
3  const NavBar = () => {
4      const navigate = useNavigate();
5
6      return (
7          <nav>
8              <button onClick={() => navigate("/")}>Home</button>
9              <button onClick={() => navigate("/about")}>About</button>
10             <button onClick={() =>
                   navigate("/contact")}>Contact</button>
11         </nav>
12     );
13 };
```

Route paramétrisée



- Routes avec des paramètres dynamiques
- Utile pour afficher des détails d'un élément spécifique
- Définition du paramètre dans le chemin avec un deux-points (`:id`)
- Récupération du paramètre avec le hook `useMatch`

Route paramétrisée



```tsx
// main.tsx, Définition de la route
{
    path: "/users/:id",
    element: <UserDetailPage />,
}

// UserDetailPage.tsx, Récupération du paramètre
const UserDetailPage = () => {
    const match = useMatch("/users/:id");
    const id = match?.params.id;
    if (!id) return <div>User ID not found</div>;
    retturn <div>User ID: {id}</div>;
};

// Autre composant, Navigation vers la route paramétrisée
navigate(`/users/${id}`);
```

Résumé



- React Router permet de gérer la navigation dans une SPA
- Les routes sont définies avec un chemin et un composant
- Les routes peuvent être imbriquées pour partager des layouts
- La navigation peut se faire avec le composant `Link` ou le hook `useNavigate`
- Les routes peuvent être paramétrisées